Ocean Wind Field Estimation using SAR imagery over Indian coastal areas for wind farm planning

Aditya Raj (25113012)
Department of Civil Engineering
Indian Institute of Technology Roorkee
Roorkee, India

Code repository: github.com/aditonu/Resnet_Wind_Products

***Abstract*—We built a working API that gives the ocean wind field (direction and speed) on a 5 km grid from Sentinel-1 SAR images. The user draws an area of up to 50 km × 50 km on a map and gives a date range; the API finds every Sentinel-1 image in that period and returns the wind vectors and a map.**

Wind direction comes from a ResNet (M64RN4, after Zanchetta & Zecchetto, 2021) that reads the wind streaks in each 5 km patch. The 180° ambiguity is removed with Copernicus Marine hourly wind, and wind speed comes from the CMOD5.N model. The network was trained on 12 Sentinel-1 scenes (including Gujarat, Kutch, Chennai and Thirupoondi) out of 31 scenes collected.

On scenes the model had not seen (scene-wise cross-validation), the mean direction error is 16.0° and the median is 10.2°. On 23 scenes the CMOD5.N speed has a bias of +0.86 m/s and an RMSD of 1.85 m/s against Copernicus Marine. Errors grow to 30–60° when the wind is weak or the image shows no clear streaks; the API marks these arrows as low quality.

***Index Terms*—Sentinel-1, SAR, wind direction, ResNet, CMOD5.N, Google Earth Engine, Copernicus Marine, offshore wind**

1. Introduction

Offshore wind farm planning needs wind maps much finer than the 12.5–25 km grid of scatterometers and weather models, especially close to the coast. Sentinel-1 SAR sees the sea surface at 10 m in any weather, day or night, so it can show the wind in that detail.

SAR does not measure the wind directly. It measures backscatter (σ^0), which depends on wind speed, wind direction and the radar angle. So the direction must be found first, from the wind streaks visible in the image, and only then can the speed be computed with a geophysical model function (GMF).

Problem statement (CEC Open Project 2.1.1). Study area: Tamil Nadu or Gujarat coast. Data: Sentinel-1. Deliverables:

Deliverable	What we made
API-based working code: user gives date and area, API gives wind vectors	Colab notebook `02_wind_field_api.ipynb`: draw a box, choose dates, get wind vectors

Deliverable	What we made
	(GeoJSON, CSV) and maps
Python / GEE code repository	GitHub repository with the Earth Engine script, Python code, notebooks and trained model
Validation results table	Section 5 and the `Validation_Results/` folder
Final report	this report

Our objectives

1. Download Sentinel-1 patches of 5 km × 5 km with Google Earth Engine.
2. Train a ResNet to find the wind direction in each patch (method of Zanchetta & Zecchetto, 2021).
3. Compute wind speed with CMOD5.N using that direction.
4. Test the model honestly, on scenes it has never seen.
5. Put everything behind a simple API that any user can run.

2. Study area and data

The API is meant for the Tamil Nadu and Gujarat coasts, but a network trained only there would see too few wind directions. So we collected 31 Sentinel-1 scenes: 4 from Indian waters (Gujarat, Kutch, Chennai, Thirupoondi) and 27 from coasts around the world with clear wind streaks.

2.1 Data sources

Data	Product	Used for
SAR images	Sentinel-1 IW GRDH, VV polarisation (`COPERNICUS/S1_GRD` in Google Earth Engine), Sentinel-1A, 1B, 1C and 1D	input to the model
Reference wind	Copernicus Marine global L4 hourly wind, 0.125° (`cmems_obs-wind_glo_phy_my_l4_0.125deg_PT1H`; covers 2007 to 20 June 2026)	training labels, 180° ambiguity, validation
Reference wind (recent dates, API only)	Copernicus Marine near-real-time L4 (`cmems_obs-wind_glo_phy_nrt_l4_0.125deg_PT1H`)	180° ambiguity in the API after June 2026
Land mask	`global-land-mask` Python package (about 1 km)	removing land pixels
Independent check	Copernicus Marine L3 scatterometer winds (ASCAT on MetOp-B/C, HY-2B/C/D, OceanSat-3)	checking the reference wind

2.2 Scenes

Each scene was exported as 10 × 10 = 100 patches of 5 km (a few scenes have fewer). Note: the scene named "Chennai" lies at 14.70–15.15° N, off Kavali in Andhra Pradesh, about 180 km north of Chennai.

In total we had 3,001 patch files; 2,795 patches remained after removing land and no-data, and 2,092 of them have a Copernicus label.

Scene	Date (UTC)	Satellite, pass	Sea patches	Copernicus wind (m/s)	Used as
Gujarat	2024-01-11	S1A, DSC	100	7.7	training
Kutch	2023-06-09	S1A, DSC	96	6.2	training
Chennai	2024-03-05	S1A, DSC	100	4.0	training
Thirupoondi	2024-11-12	S1A, DSC	67	5.1	training
Netherland	2024-01-04	S1A, ASC	99	8.4	training
Brazil	2024-10-01	S1A, DSC	96	7.9	training
Chile	2024-02-02	S1A, DSC	95	7.8	training
Perth	2024-02-02	S1A, DSC	98	7.4	training
Cape Peninsula	2024-03-12	S1A, ASC	93	6.2	training
Tehuantepec	2025-04-04	S1C, DSC	100	5.8	training
Vietnam	2024-02-07	S1A, ASC	100	4.3	training
Lima	2024-01-03	S1A, DSC	96	4.1	training
Durban	2024-07-06	S1A, ASC	100	7.8	test only
Vancouver 2	2020-11-04	S1B, ASC	94	7.5	test only
Bahia	2024-09-07	S1A, DSC	98	5.3	test only
Oaxaca	2023-12-01	S1A, ASC	100	4.5	test only
Oman	2024-05-23	S1A, ASC	61	4.2	test only
Vancouver	2020-01-09	S1B, ASC	94	3.7	test only
San Diego	2024-01-03	S1A, ASC	100	3.5	test only
Timor	2021-04-08	S1A, DSC	7	3.4	test only
NW Madagascar	2024-10-12	S1A, DSC	100	3.1	test only
Mistral	2019-08-01	S1B, DSC	98	2.4	test only
Vegan City	2023-04-04	S1A, DSC	100	1.5	not used (wind too weak)
Aleutians, Chukchi, Pohang, Busan, SC_SK,	2026-10-03 to 2026-10-12	S1C / S1D	52–100 each, 703 in	—	not used (no Copernicus label yet)

Scene	Date (UTC)	Satellite, pass	Sea patches	Copernicus wind (m/s)	Used as
Jeju, Prince William Sound, Eureka			total		

"Copernicus wind" is the scene mean at the patch centres. Why a scene is used only for testing is explained in Section 4.1.

3. Methodology

The same processing is used for training and inside the API, so the model always sees data prepared in exactly the same way.

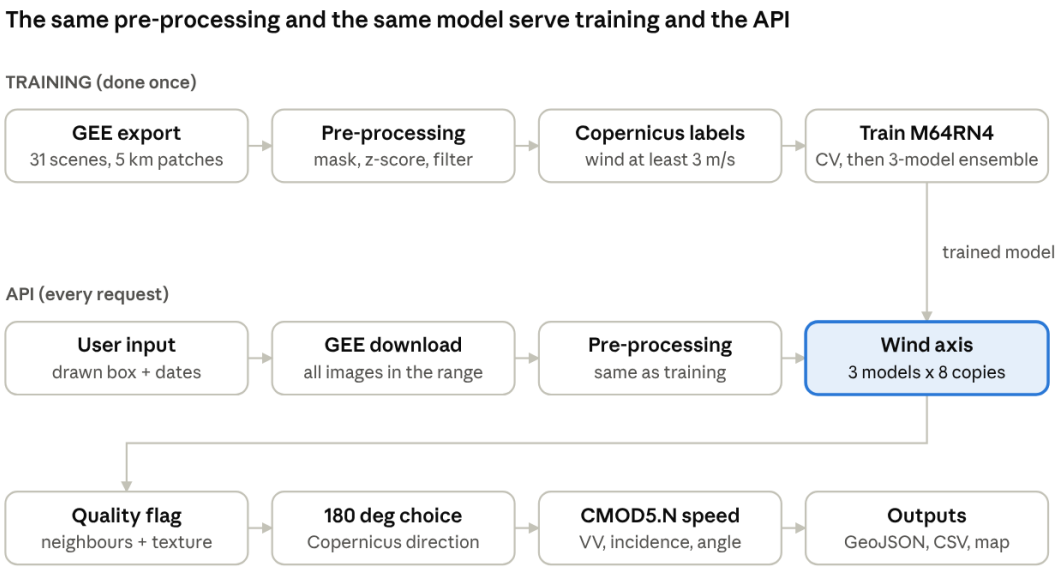


Figure 1 · processing pipeline: training (top) and API (bottom)

The trained model from the top row is the only link between training and the API; everything else in the API repeats the training steps on new images.

3.1 Downloading the patches (Google Earth Engine)

The Earth Engine script `gee/s1_patch_export.js` does the following for each area and date range:

1. Selects Sentinel-1 images with `instrumentMode = IW`, VV polarisation and 10 m pixels (GRDH) that touch the area.
2. Groups the image slices into one acquisition (same satellite, date, pass and relative orbit) and keeps the one that covers most of the area.
3. Converts VV from dB to linear power, averages the 10 m pixels to 100 m on a UTM grid centred on the area, and converts back to dB. Averaging in linear units keeps the true mean backscatter.
4. Cuts the 50 km × 50 km area into 10 × 10 patches of 5 km × 5 km (50 × 50 pixels).

5. Writes one GeoTIFF per patch with three bands: VV (dB), incidence angle (from the `angle` band, degrees) and radar look azimuth (degrees). Missing pixels are −9999.

Earth Engine has no per-pixel look-azimuth band, so we use look azimuth = platform heading + 90° (Sentinel-1 looks to the right). If the heading is missing, −12° (ascending) or −168° (descending) is used.

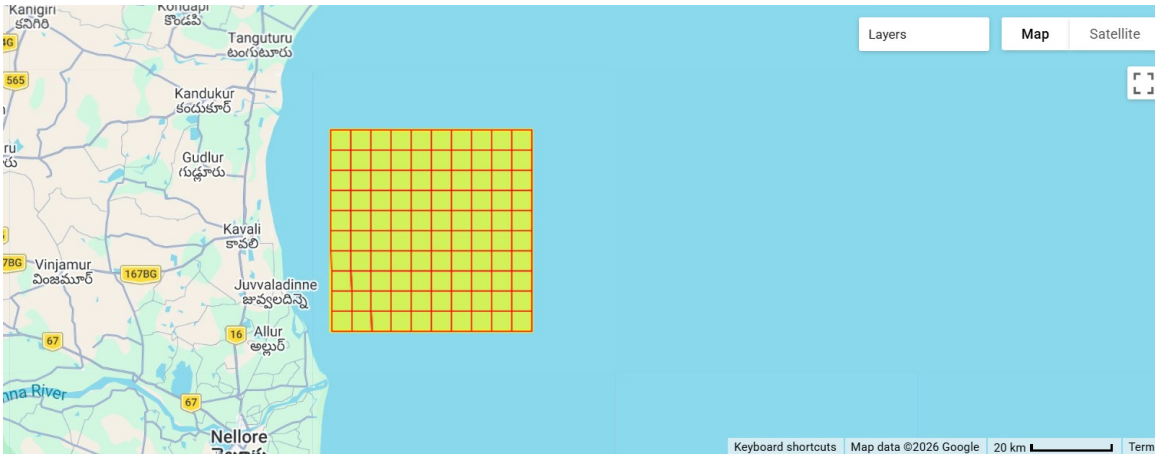


Figure 2. The 10 × 10 patch grid (5 km cells) drawn by the Earth Engine script for the "Chennai" scene box (80.20–80.66° E, 14.70–15.15° N).

3.2 Pre-processing of each patch

1. **Land mask:** pixels on land (global-land-mask) or with no data are marked bad.
2. **Remove bad patches:** a patch with more than 25% bad pixels is dropped (as in the paper).
3. **Fill small gaps:** the remaining bad pixels take the value of the nearest sea pixel plus small Gaussian noise (σ = 0.5 in z-score units).
4. **Normalise:** each patch is converted to a z-score with the mean and standard deviation of its own sea pixels.
5. **Low-pass filter:** a Gaussian filter with σ = 1.5 pixels (150 m), then a new z-score. This removes swell and speckle (waves shorter than about 600 m) and keeps the kilometre-scale wind streaks. We chose σ = 1.5 after testing no filter, 1.5 and 2.0 pixels in cross-validation.
6. **Values kept for the speed model:** the mean VV backscatter (averaged in linear units, in dB), the median incidence angle and the median look azimuth of the patch.

3.3 Wind labels (Copernicus Marine)

For each scene, one small Copernicus Marine file is downloaded around the patches (0.375° margin) for the two hours around the acquisition. The wind is interpolated linearly in time and bilinearly in space to each patch centre; near the coast, where the grid has no value, the nearest valid cell within 0.25° is used. Direction is the meteorological "from" direction, clockwise from north:

θ (from) = (270° − atan2(v, u)) mod 360°, and speed S = √(u² + v²)

Only patches with wind speed of at least 3 m/s are used for training and testing, because weaker wind leaves almost no streaks. This left 1,813 of the 2,092 labelled patches.

3.4 Wind direction model: ResNet M64RN4

We use the M64RN4 network of Zanchetta & Zecchetto (2021): 4 residual blocks with 64 filters each, followed by dense layers of 512, 128 and 32 units and one output (the wind angle). It has 627,393 parameters (Figure 3).

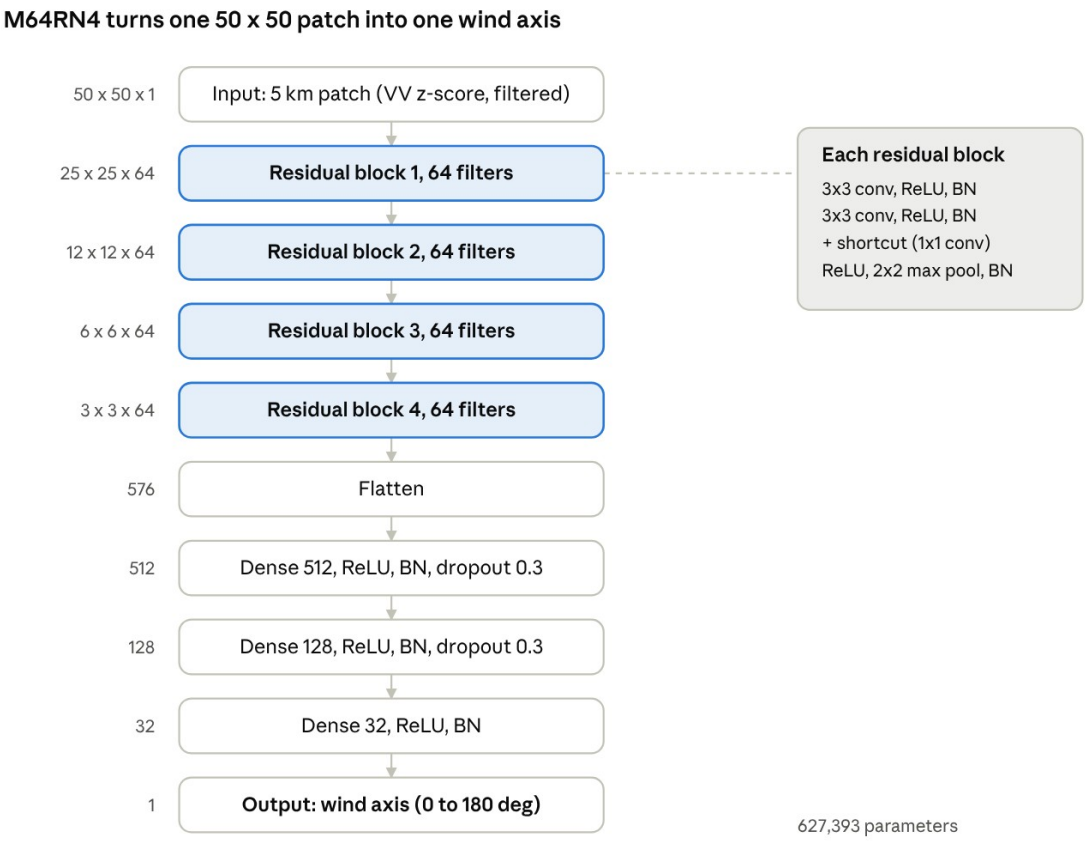


Figure 3 · M64RN4 network as used in this project (sizes after each layer on the left)

- Each residual block: 3×3 convolution → ReLU → batch normalisation, twice; a shortcut is added (1×1 convolution when the channel number changes); then ReLU, 2×2 max pooling and batch normalisation. The 50 × 50 patch becomes 3 × 3 × 64 after four blocks.
- Dropout of 0.3 after the 512 and 128 dense layers (our addition, to reduce over-fitting on few scenes).

A streak shows the wind axis but not its sense (wind from north and from south look the same). So the loss only measures the error of the axis:

Loss = 1 − cos²(θ predicted − θ true)

Training setting	Value
Optimiser	Adam, learning rate 0.001, weight decay 0.0001
Learning-rate decay	halved after 5 epochs without improvement
Batch size	64

Training setting	Value
Epochs	maximum 120, early stopping after 15 epochs without improvement
Augmentation	8 rotations and flips of every patch (90°, 180°, 270°, flips, transposes); the label angle is rotated or mirrored with the image
Final model	ensemble of 3 networks; each uses 2 different scenes for early stopping and the other 10 for training
Hardware and time	Google Colab, NVIDIA T4 GPU; final training took 7.9 minutes

The three final networks stopped at epochs 23, 25 and 19 with validation losses of 0.217, 0.103 and 0.036. They were trained on 21 October 2026 and are saved in `models/final_model/`.

3.5 From patch to wind vector

1. **Direction axis:** each of the 3 networks predicts the axis for all 8 rotated and flipped copies of the patch; each prediction is turned back to the original frame and the 24 values are averaged as axes (mod 180°). Their agreement gives a confidence from 0 to 1.
2. **Quality flag:** an arrow is "good" when (a) its axis is within 20° of the mean axis of its neighbours (patches within 7.6 km), and (b) the wind-streak texture is strong: the anisotropy of the 2D Fourier spectrum in the 0.8–2.5 km band (patch and neighbours together) is at least 0.20. Otherwise it is "low". If the mean anisotropy of the whole area is below 0.21, the map shows a warning.
3. **180° ambiguity:** of the two possible directions (axis and axis + 180°), the one closer to the Copernicus Marine direction is chosen.
4. **Wind speed:** CMOD5.N (from the `xsarsea` library) is inverted: we search wind speeds from 0.2 to 35 m/s in steps of 0.05 m/s for the one whose model backscatter matches the measured mean VV of the patch, using its incidence angle and the relative angle

φ = θ (from) − radar look azimuth

5. **Vector:** u and v are computed from speed and direction, and land is shown grey on the map.

The thresholds of the quality flag were set from the cross-validation errors of the first model: "good" arrows were 60% of patches with a mean error of 13.6° (median 9.9°); "low" arrows had a mean error of 36°.

4. Validation method

Every direction score in this report comes from scenes the model did not see during training.

4.1 Which scenes are used for training

Before training, we measured the streak texture of every scene with a 2D Fourier transform (no neural network): the anisotropy of the 0.8–2.5 km band and the angle between the streaks and the Copernicus wind. A scene goes to training only when its streaks are clear (anisotropy about 0.18 or more) and follow the reference wind. The other scenes are kept for testing only, so the network does not learn wrong examples, but we still see how it does on hard cases.

Test-only scene	Streak anisotropy	Streak angle vs Copernicus	Reason
NW Madagascar	0.08	21°	almost no texture
Bahia	0.08	43°	almost no texture
Vancouver 2	0.09	21°	almost no texture
Oman	0.11	65°	weak texture, streaks not along the wind
Vancouver	0.13	64°	weak texture, streaks not along the wind
San Diego	0.15	48°	weak texture, streaks not along the wind
Mistral	0.18	18°	weak wind (2.4 m/s mean)
Timor	0.18	18°	only 7 patches
Oaxaca	0.24	59°	clear texture, but 59° away from the reference wind
Durban	0.25	28°	streaks 28° away from the reference wind

The 12 training scenes have anisotropy from 0.19 to 0.51, and their streak angle is within 22° of the Copernicus wind (most within 8°).

4.2 Scene-wise cross-validation

Neighbouring patches of one scene have almost the same wind. If patches were split at random, the test patches would be "neighbours" of training patches and the score would look better than it is. So we split by scene:

- The 12 training scenes were sorted by wind axis and placed into 5 folds in turn, so each fold has different directions.
- For fold k: its scenes are the test set, the scenes of fold k+1 are used for early stopping, and the rest are used for training.
- The 10 test-only scenes were predicted by the 5 fold models together; none of them was trained on these scenes.

We moved to this method because our first version used a single split with 3 test scenes, and running it twice with the same settings changed the Netherland error from 44° to 35°. One split was too noisy to compare settings.

4.3 Error measures

Measure	Meaning
Direction error	smallest angle between the predicted axis and the Copernicus wind axis (−90° to +90°); the 180° sense is not counted
MAE, median	mean and median of the absolute direction error
Within 15°	share of patches with error of 15° or less

Measure	Meaning
Speed bias, RMSD	CMOD5.N speed minus Copernicus speed: mean, and root-mean-square

CMOD5.N is a fixed physical model and was not trained, so its speed comparison is fair on every scene, including training scenes.

4.4 Independent checks

- **Direction against the image itself** (`check_direction.py`): the FFT streak direction of each patch is compared with the network and with Copernicus. This tells a code error apart from a real difference between the streaks and the wind.
- **Reference against satellite scatterometers** (`scat_compare.py`): Copernicus L4 is compared with near-simultaneous ASCAT, HY-2 and OceanSat-3 winds, to check whether the reference itself is right near the coast.

5. Results

On the 12 training scenes, tested one fold at a time, the mean direction error is 16.0° and the median is 10.2°; on the 10 weak-texture test scenes it is 43.0°.

5.1 Wind direction summary

Group	Scenes	Patches	Mean error (°)	Median error (°)	Within 15° (%)	Error of scene-mean axis (°)
Training scenes (cross-validation)	12	1,135	16.0	10.2	63	8.4
Test-only scenes	10	678	43.0	41.1	18	38.0
All	22	1,813	26.1	17.1	46	21.8

A random guess gives a mean error of 45°. Averaging over the 8 rotated copies (test-time augmentation) lowered the mean error per scene from 17.3° to 16.2°. Our first model, trained on 10 scenes, had 15.9° mean and 10.5° median error on its cross-validation; the new model keeps that level while also covering Kutch and Tehuantepec.

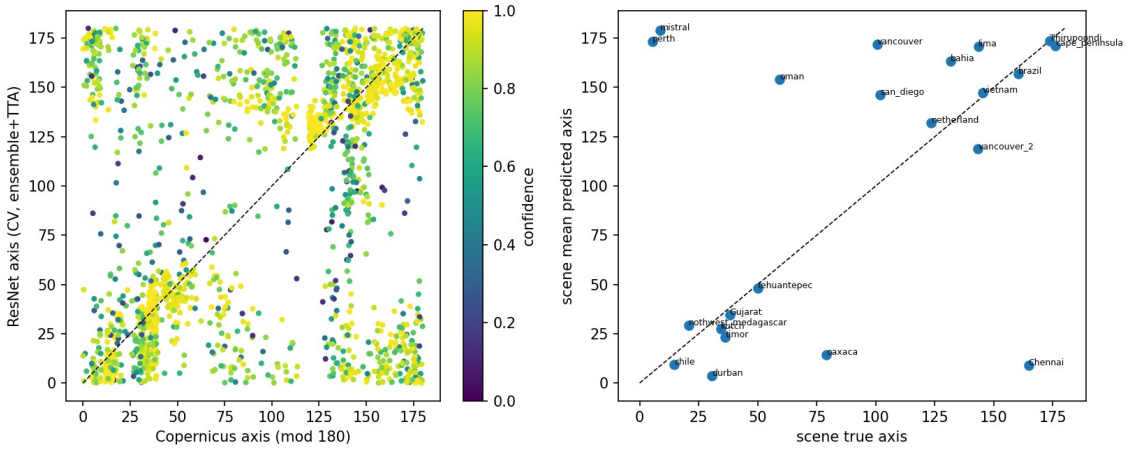


Figure 4. Cross-validation. Left: network axis against Copernicus axis for every patch (colour = confidence); points in the top-left and bottom-right corners are also correct, because 0° and 180° are the same axis. Right: mean axis of each scene; training scenes lie near the line, test-only scenes such as Oaxaca, Oman and Vancouver lie far from it.

5.2 Validation results per scene

Direction errors are from cross-validation (scene not seen in training). Speed is CMOD5.N with our direction, compared with Copernicus Marine.

Scene	Used as	Patches (≥ 3 m/s)	Copernicus wind (m/s)	Dir. mean error (°)	Dir. median error (°)	Within 15° (%)	Speed bias (m/s)	Speed RMSD (m/s)
Brazil	training	96	7.9	8.1	7.1	88	+1.38	1.98
Netherland	training	99	8.4	9.1	6.6	85	+1.21	1.66
Chile	training	95	7.8	9.2	9.4	89	+0.66	1.27
Kutch (Gujarat)	training	96	6.2	9.8	6.4	75	+2.38	2.51
Tehuantepec	training	100	5.8	10.8	7.6	78	−0.39	0.70
Gujarat	training	100	7.7	14.1	8.9	68	−0.22	1.00
Cape Peninsula	training	93	6.2	15.3	7.0	65	+0.09	1.81
Vietnam	training	100	4.3	18.7	10.3	57	+1.23	1.71
Perth	training	98	7.4	19.6	15.0	50	+0.54	1.49
Thirupoondi (Tamil Nadu)	training	67	5.1	20.3	18.9	43	+1.62	1.75
Chennai (off	training	100	4.0	24.3	23.2	26	+1.12	1.17

Scene	Used as	Patches (≥ 3 m/s)	Copernicus wind (m/s)	Dir. mean error (°)	Dir. median error (°)	Within 15° (%)	Speed bias (m/s)	Speed RMSD (m/s)
Andhra Pradesh)								
Lima	training	91	4.1	35.0	35.8	24	+1.94	2.38
Timor	test only	7	3.4	18.7	17.7	43	+0.34	0.47
Mistral	test only	30	2.4	21.5	19.9	43	+1.45	2.13
NW Madagascar	test only	56	3.1	31.6	22.8	34	+0.52	1.13
Durban	test only	100	7.8	31.9	30.3	25	+1.08	1.90
Vancouver 2	test only	94	7.5	33.1	32.0	24	−0.21	2.11
Bahia	test only	95	5.3	38.1	36.5	25	+2.23	3.02
San Diego	test only	69	3.5	44.6	44.5	1	+2.50	2.59
Oaxaca	test only	90	4.5	53.2	56.2	4	−1.44	1.75
Oman	test only	61	4.2	62.3	72.2	8	+1.59	2.14
Vancouver	test only	76	3.7	65.6	70.5	4	−0.06	1.88

The same table is in `Validation_Results/validation_summary.csv`. Speed values use all sea patches of the scene, including those below 3 m/s.

5.3 Wind speed summary

Group	Patches	Speed bias (m/s)	Speed RMSD (m/s)
All 23 labelled scenes	2,092	+0.86	1.85
Training scenes	1,140	+0.94	1.69
Other scenes	952	+0.77	2.03
Only arrows flagged "good"	1,212	+0.97	1.71

58% of all sea patches were flagged "good": 80% on the training scenes and 32% on the others. For the first model we also found that the speed bias depends on the wind: about +1.8 m/s below 3 m/s and +0.3 m/s at 8–12 m/s. Using the Copernicus direction instead of ours in CMOD5.N gave a bias of +0.79 m/s and RMSD of 1.46 m/s, so most of the speed error comes from CMOD5.N and the reference, not from our direction.

5.4 What the results show

- **Clear streaks give good directions.** Most scenes with wind of about 6 m/s or more and clear streaks have mean errors of 8–15°, including both Gujarat scenes (Kutch 9.8°, Gujarat 14.1°); Perth (19.6°) is the exception.
- **Weak texture gives poor directions.** Eight of the ten test-only scenes have mean errors of 32–66°; they have little texture or streaks that do not follow the reference wind. The quality flag catches most of these: only 32% of the patches outside the training scenes are "good".
- **Chennai and Lima have a steady offset.** Their scene-mean axis is 24° and 27° away from Copernicus. The independent checks showed that the network follows the streaks in the image within 1–4°, and that Copernicus L4 agrees with near-simultaneous scatterometer winds within about 5–10°. So the offset is real: near the coast the streaks are turned against the surface wind (11–20° in the scenes we checked). It is not a code error.
- **CMOD5.N speed is a little high** (+0.86 m/s on average), most at low wind; Kutch (+2.38 m/s) is the largest positive bias among training scenes.

5.5 Wind field maps (Indian scenes)

The maps below are made by the final model. These three scenes were used in training, so their directions here are in-sample; their unseen-scene errors are the cross-validation numbers in Section 5.2.

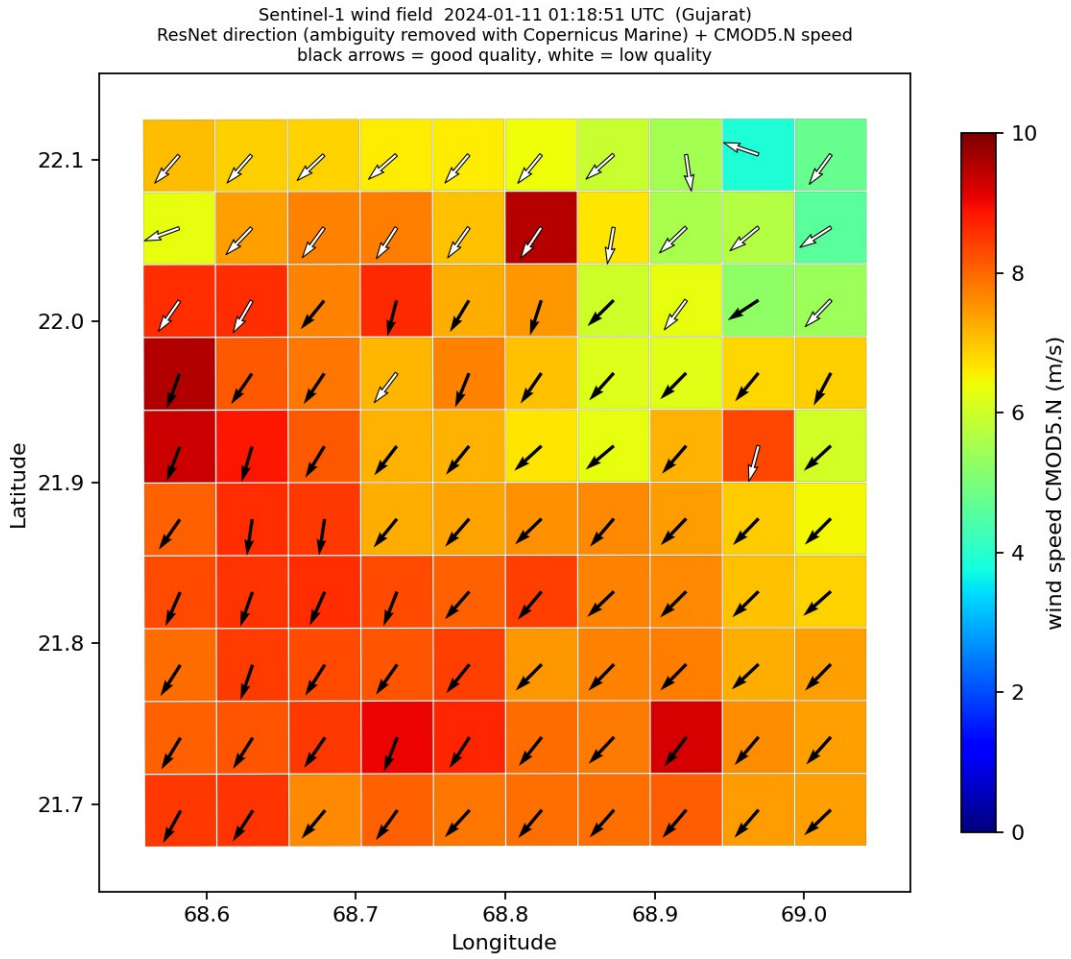


Figure 5. Gujarat, 11 January 2024, 01:18 UTC: north-east wind of 6–10 m/s; 74 of 100 arrows are "good" (black).

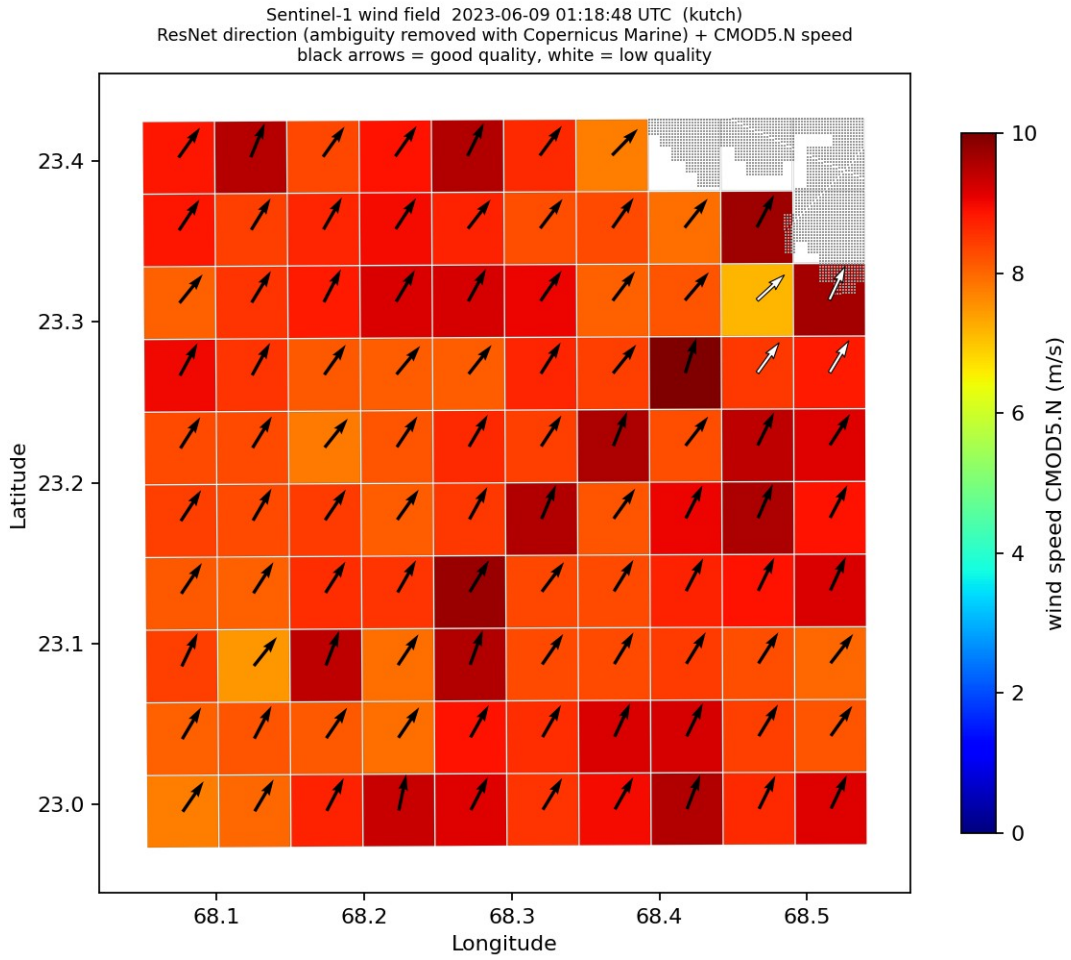


Figure 6. Kutch (Gujarat), 9 June 2023, 01:18 UTC: strong south-west wind of 7–10 m/s; land at the top right is masked.

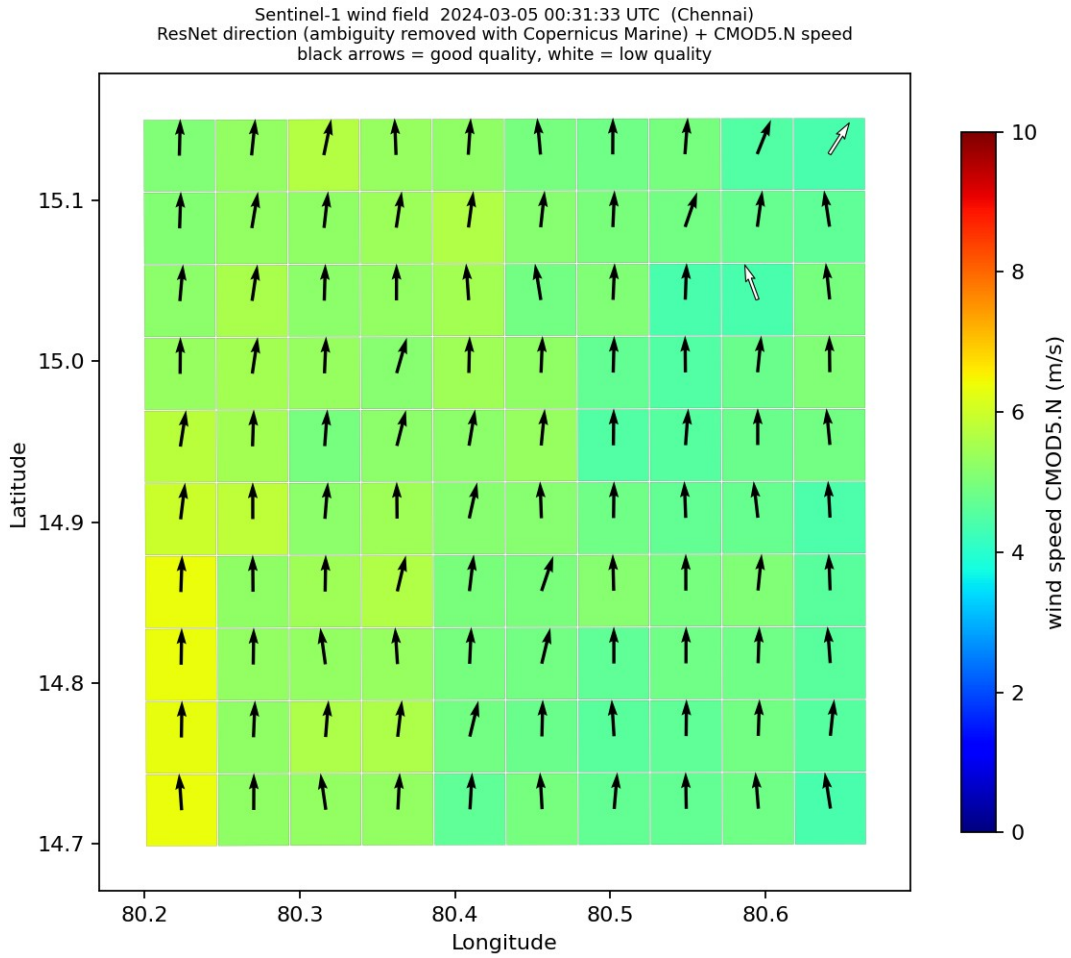


Figure 7. "Chennai" scene (off Kavali, Andhra Pradesh), 5 March 2024, 00:31 UTC: wind of about 4–6 m/s from the south; nearly all arrows are "good", but the streaks are about 20° off the Copernicus direction (Section 5.4).

6. The API

The API runs in Google Colab: the user draws a box and picks two dates, and gets the wind vectors and a map for every Sentinel-1 image in that period. No installation and no server are needed; the notebook downloads the code and the trained model (3 files of 2.5 MB) from GitHub.

6.1 How a user runs it

1. **Setup** (once per session): enter the Earth Engine project id and press ▶️. The cell downloads the repository, installs the libraries, signs in to Google Earth Engine and Copernicus Marine, and loads the model.
2. **Step 1 — draw the area:** a map (satellite or street view) with a rectangle tool. The box stops growing at 50 km × 50 km while drawing, and its size is shown live. The user never types coordinates.
3. **Step 2 — dates:** start and end date (UTC, both included, at most 61 days). The cell shows one map per Sentinel-1 image.

4. **Step 3 — download:** all results as one zip file.

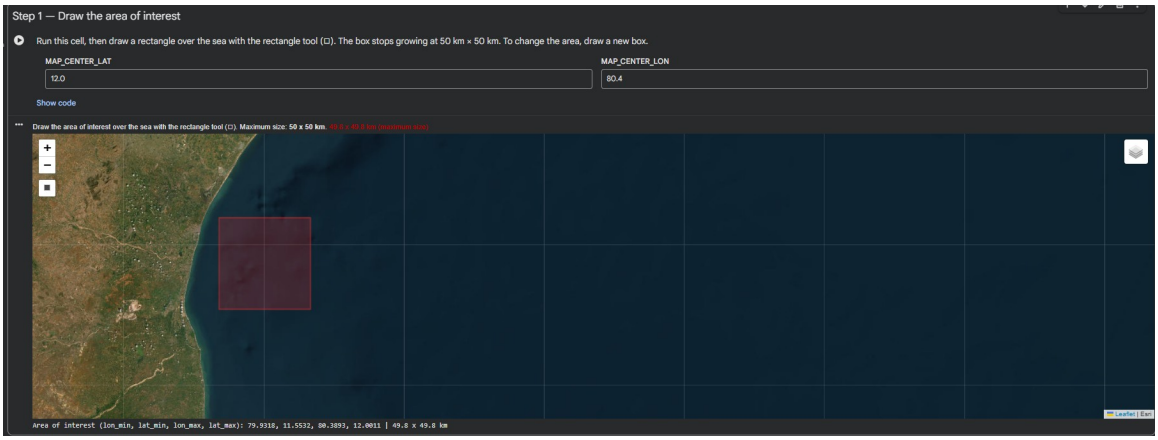


Figure 8. Step 1 in Colab: a box drawn off the Tamil Nadu coast. It stopped at the 49.8 km × 49.8 km limit (red text at the top), and its corners are printed below the map.

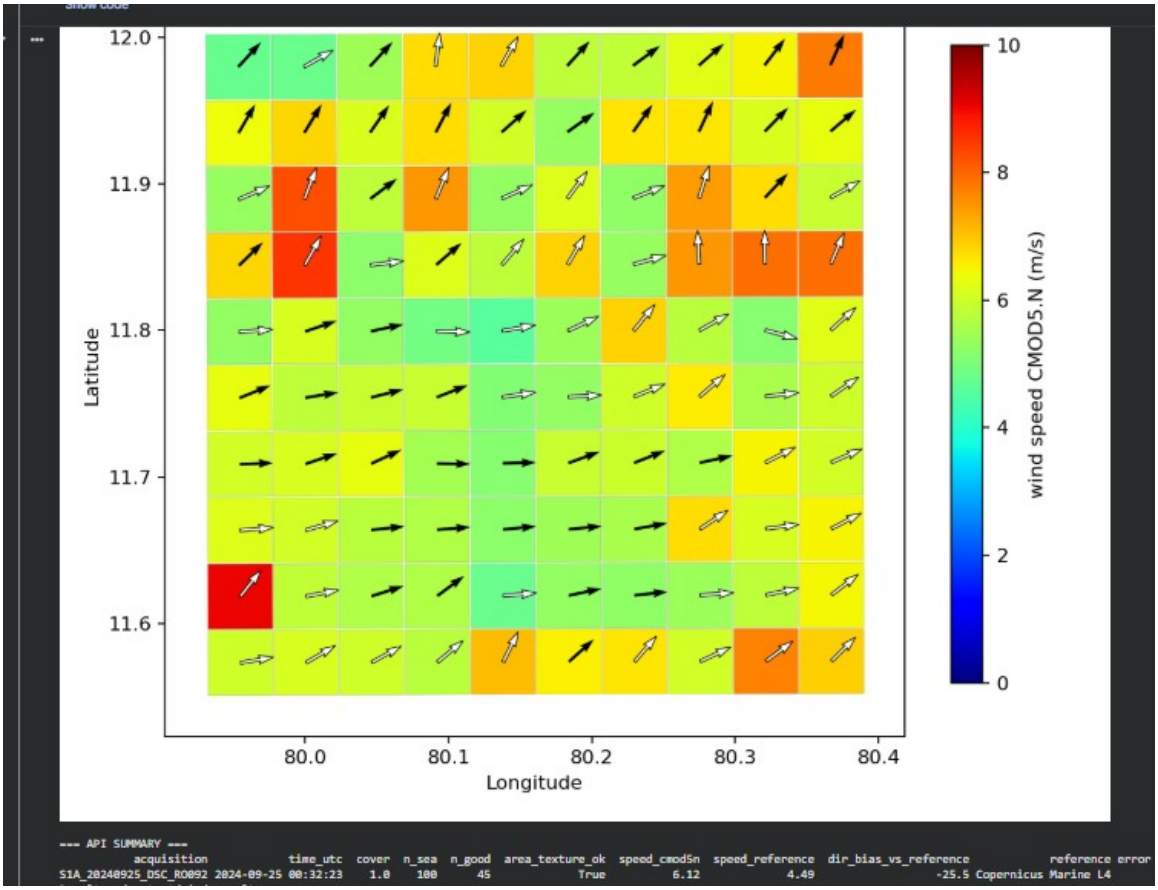


Figure 9. Step 2 output for that box (25 September 2024): the wind map and the API summary line (100 sea patches, 45 good, reference Copernicus Marine L4).

6.2 What happens inside

1. The box is checked (5 km to 50 km a side) and a UTM grid of 5 km patches is placed on it (up to 10 × 10).
2. Earth Engine is searched for Sentinel-1 IW GRDH VV images in the date range. Images covering at least 50% of the box are processed (up to 6 per request). If none is found, the nearest dates within 15 days are listed.
3. Each image is processed exactly like the training data (Section 3.1) and downloaded with `ee.data.computePixels`, then cut into patch GeoTIFFs.
4. The patches go through Section 3.2 and 3.5: model, quality flag, Copernicus Marine reference (reprocessed product, near-real-time product for recent dates), CMOD5.N speed and the map.

6.3 Outputs

File	Content
`wind_vectors.geojson`	one point per 5 km sea patch: `wind_from_deg`, `wind_speed_ms`, `u_ms`, `v_ms`, `quality`, `ambiguity_removed`, `time_utc`
`wind_field_*.csv`	the same vectors with all intermediate values (axis, confidence, texture, incidence, backscatter, Copernicus wind)
`map_*.png`	arrows coloured by speed on a map; black = good, white = low quality; land grey
`summary_*.json`	image details and comparison with Copernicus Marine

GeoJSON opens directly in QGIS or any GIS software. For other programs the same API is one Python call: `wind_app.get_wind_field(box, start_date, end_date)`.

6.4 Example run: Tamil Nadu coast off Cuddalore

Box 79.84–80.30° E, 11.39–11.84° N; Sentinel-1A image of 25 September 2024, 00:32 UTC (descending, relative orbit 92), not used in training.

Result	All 100 sea patches	57 "good" patches
Direction error vs Copernicus (mean)	19.8°	11.9°
Direction bias	−14.3°	—
Mean speed: CMOD5.N / Copernicus	6.25 / 4.40 m/s	—
Speed bias / RMSD	+1.85 / 2.12 m/s	—

The area texture (anisotropy 0.26) was above the warning level. The "good" arrows match the reference about as well as in the cross-validation (median 10.2°).

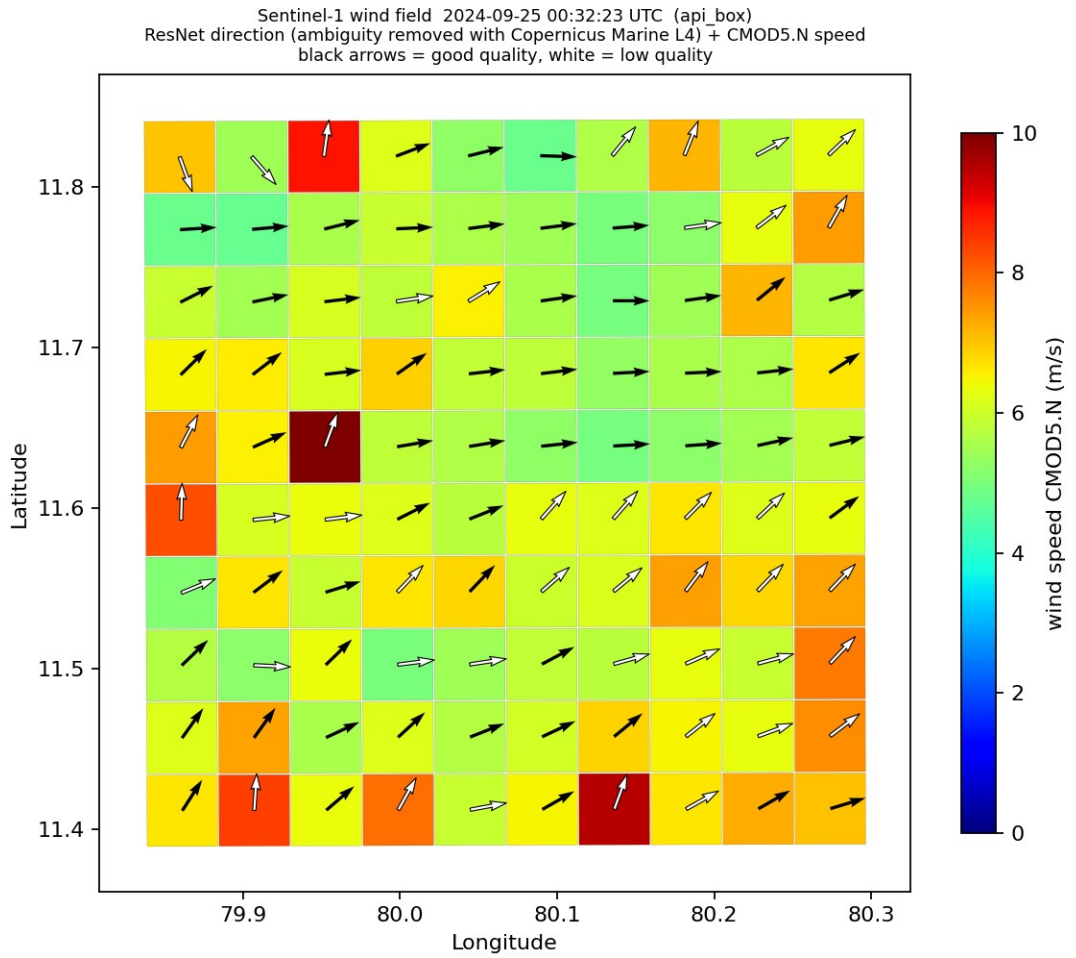


Figure 10. API output off Cuddalore (Tamil Nadu), 25 September 2024: wind from the west and south-west, 4–8 m/s.

A second test off the Kerala coast (13 July 2024, monsoon) shows the warning at work: area anisotropy was only 0.18, just 18 of 97 sea patches were "good", and what look like monsoon rain cells gave patches of 14–16 m/s. The map printed "weak SAR texture — trust only black arrows".

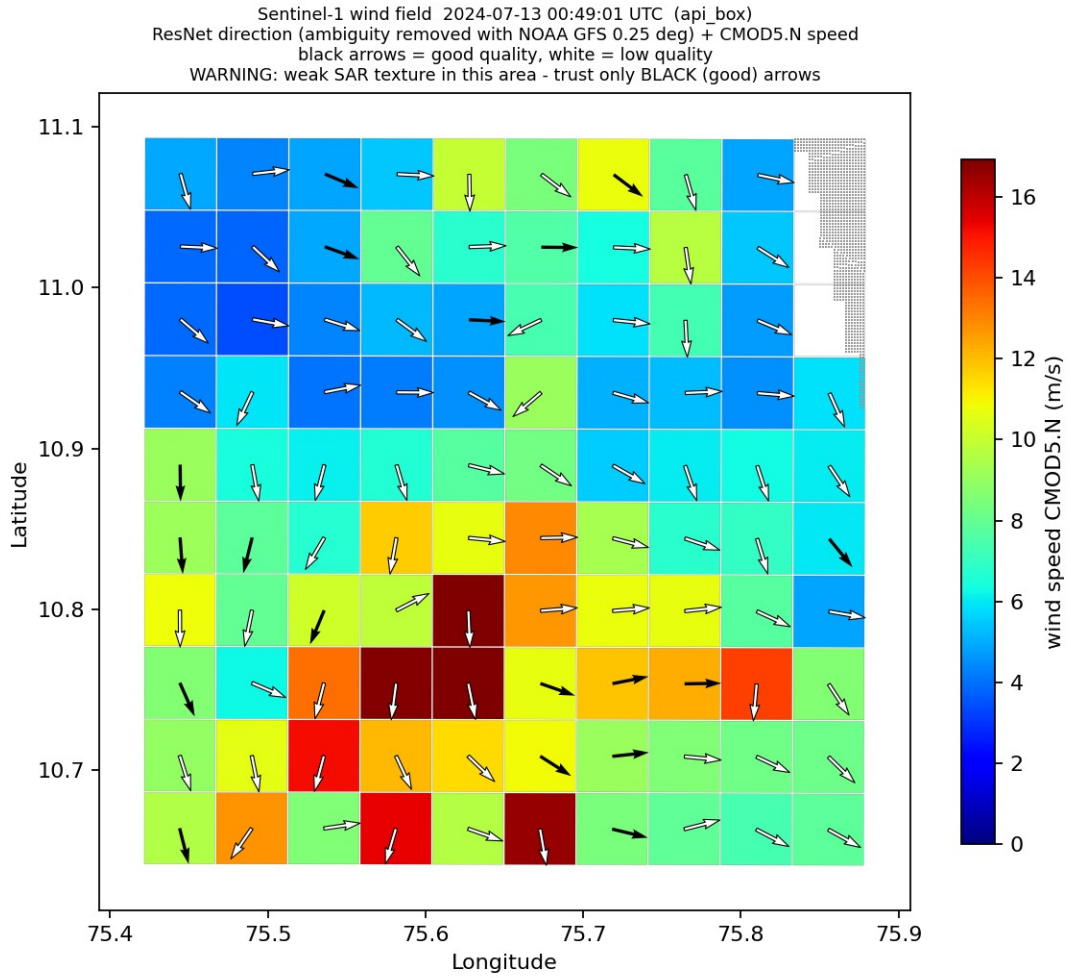


Figure 11. API output off Kerala, 13 July 2024 (monsoon): mostly white "low" arrows and a weak-texture warning in the title.

7. Problems faced and how we solved them

Problem	Cause	Solution
Scores changed a lot between identical runs (Netherland 44°, then 35°)	only one split with 3 test scenes	scene-wise cross-validation, an ensemble of models and test-time augmentation
On some scenes the network followed swell, not the wind streaks	swell (waves of 220–600 m) is often stronger than the streaks	Gaussian low-pass filter (σ = 1.5 px) before the network
Some scenes taught wrong directions	weak texture or streaks not along the reference wind	FFT texture check before training; such scenes used for testing only; quality flag on every arrow
Direction error of about 20° near Chennai and Palk Bay;	streaks near the coast are turned against the surface	checked the network against the image FFT (1–4°) and Copernicus

Problem	Cause	Solution
looked like a code bug	wind	against scatterometers (5–10°): the offset is physical, the code is correct
Training failed in Colab: `unexpected keyword 'dropout'`	an old copy of `train_resnet.py` in Google Drive	replaced the file and restarted the runtime
The dataset file could only be opened with `allow_pickle`	text columns saved as Python objects	saved them as plain text arrays
QC report showed a wrong mean direction for Thirupoondi	directions near 0°/360° averaged as normal numbers	circular mean of directions
A Sentinel-1C scene (100 files) was skipped	file-name check allowed only S1A and S1B	accept S1A to S1D
8 scenes from October 2026 got no labels	the reprocessed Copernicus product ends on 20 June 2026	left out of training; the API uses the near-real-time product for recent dates
Map tiles did not load in Colab (error 403)	OpenStreetMap tiles were blocked	our own Leaflet map with Esri and CARTO tiles; the box is limited to 50 km while drawing
A single date often returned no image	Sentinel-1 passes a place only every few days	the user gives a date range; nearest dates are listed when none is found
The API marked a Chennai image as "unseen" although it was a training scene	training scenes were stored by folder name only	scene ids are saved in the model's `config.json`
A friend would need our 5–6 GB Drive folder to run the API	code, model and data were mixed in one folder	the API needs only the code and the 8 MB model, downloaded from GitHub in Setup
Old code kept running in Colab after an update	Python keeps loaded modules in memory	Setup reloads the project code every time

8. Limitations and future work

Limitations

- **Weak wind or weak texture:** below about 5 m/s, or without clear streaks, direction errors are 30–60°. These arrows are flagged "low" (white), but they are still drawn.
- **Few Indian training scenes:** only 4 of the 12 training scenes are from Indian waters (2 Gujarat, 1 Tamil Nadu, 1 Andhra Pradesh).

- **Missing monsoon direction:** no training scene has a wind axis between 60° and 120°. This is the axis of the south-west monsoon (wind from the west or south-west) on these coasts. Rotating the images in training covers it only partly.
- **Coastal offset:** near some coasts the streaks are turned 20–27° against the surface wind (Chennai, Lima); the network follows the streaks.
- **Reference wind:** Copernicus L4 has a 0.125° grid (about 14 km) and is partly model-based, so the labels are smoother than the SAR image. The 180° choice is wrong wherever the reference is more than 90° off.
- **Speed:** CMOD5.N gives about +0.9 m/s more than Copernicus, more at low wind; rain cells in the monsoon can give false high speeds.
- **Look azimuth:** one value per image slice (heading + 90°), not per pixel.
- **Use:** each user needs a free Earth Engine project and a Copernicus Marine account; one box of up to 50 km × 50 km and up to 6 images per request; a few minutes per image.

Future work

1. Add 10–15 Tamil Nadu and Gujarat scenes with wind of at least 6 m/s, especially June–September (south-west monsoon) and October–December (north-east monsoon), and retrain.
2. Validate with in-situ buoy winds near the coast.
3. Correct the CMOD5.N bias with a coastal calibration from the validation data.
4. For "low" quality patches, smooth with the neighbours or fall back to the reference direction.

9. Conclusion

We delivered a working, easy-to-use API that turns a drawn box and a date range into a 5 km wind field from Sentinel-1, with direction from a ResNet M64RN4 ensemble and speed from CMOD5.N.

Tested honestly on unseen scenes, the direction error is 16.0° on average and 10.2° in the median, and 8–15° on most scenes with clear streaks, including both Gujarat scenes. The speed is within about 1.9 m/s RMSD of Copernicus Marine, with a small positive bias. Where the SAR image has little information (weak wind, rain, no streaks) the API says so with its quality flag instead of hiding it.

The main next step is more Tamil Nadu and Gujarat monsoon scenes in training; retraining (cross-validation and final model) takes about half an hour on a free Colab GPU with the notebook in the repository.

10. Code and data availability

All code, the trained model and the validation results are on GitHub: github.com/aditonu/Resnet_Wind_Products. The README has an **Open in Colab** button for the API and a step-by-step `SETUP_GUIDE.md`.

Folder / file	Content
`notebooks/02_wind_field_api.ipynb`	the API
`notebooks/01_training_and_validation.ipynb`	dataset, cross-validation, final training, validation
`gee/s1_patch_export.js`	Earth Engine script for the training patches
`src/`	Python code (dataset, training, inference, API,

Folder / file	Content
	checks)
`models/final_model/`	the 3 trained M64RN4 networks and `config.json`
`Validation_Results/`	validation tables, cross-validation plot and maps

The 3,001 patch GeoTIFFs (about 5–6 GB with the working files) are not in the repository; they can be made again with the Earth Engine script and the scene dates in Section 2.2.

References

1. Zanchetta, A., Zecchetto, S. (2021). Wind direction retrieval from Sentinel-1 SAR images using ResNet. *Remote Sensing of Environment*, 253, 112178. sciencedirect.com
2. Hersbach, H. (2010). Comparison of C-band scatterometer CMOD5.N equivalent neutral winds with ECMWF. *Journal of Atmospheric and Oceanic Technology*, 27(4), 721–736.
3. He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep residual learning for image recognition. *IEEE CVPR 2016*, 770–778.
4. Gorelick, N., et al. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202, 18–27.
5. ESA. Wind Field Estimation, SNAP Microwave Toolbox help, version 13.0. step.esa.int
6. Copernicus Marine Service. Global ocean hourly L4 sea surface wind, datasets `cmems_obs-wind_glo_phy_my_l4_0.125deg_PT1H` and `cmems_obs-wind_glo_phy_nrt_l4_0.125deg_PT1H`; L3 scatterometer winds. marine.copernicus.eu
7. Copernicus Sentinel-1 data (ESA), Earth Engine collection `COPERNICUS/S1_GRD`.
8. xsarsea (CMOD5.N implementation). github.com/umr-lops/xsarsea
9. global-land-mask. github.com/toddkarin/global-land-mask
10. PyTorch. pytorch.org